

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO5: Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code. (BL3)

Assessment Tool Weight-age: 40%

QUESTIONS: 5

Total Marks:50

Annexure A

SIMULATION TASK

Code of Conduct:

I P.Karpaga shree(192424042) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

P.Karpaga shree

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts

Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

Annexure A

SIMULATION TASK

QUESTIONS

1. Given the three-address code sequence:

$t1 = x * 1$

$t2 = t1 + y$

$t3 = t2 - 0$

$t4 = t3 * 0$

if $t4 \neq 0$ goto L1

$t5 = y + y$

Simulate the **peephole optimization** process on this sequence. Produce the optimized instruction sequence and justify each optimization step applied, including algebraic simplification, constant folding, and dead code elimination.

2. Partition the following intermediate code into **basic blocks** and construct the corresponding **control flow graph**. Identify all leader statements and mention the rules used to identify them:

(1) $x = a + b$

(2) $y = x * c$

(3) if $y > 50$ goto 6

(4) $z = y - d$

(5) goto 7

(6) $z = y + d$

(7) $w = z * 2$

3. Construct the **DAG** for the following basic block:

$p = a + b$

$q = a + b$

$r = p * q$

$s = a + b$

$t = r + s$

Using the DAG, identify all **common sub-expressions**. Then simulate a simple code generator to produce optimized target code and explain the register allocation decisions at each step.

4. For a target machine with two registers **R0** and **R1**, simulate the working of a **simple code generator** for the code:

$t1 = a + b$

$t2 = c * d$

$t3 = t1 - t2$

$t4 = e + f$

$t5 = t3 / t4$

Show the **register descriptor** and **address descriptor** after each instruction is processed.

5. For a **RISC-style target machine** with instructions **LOAD**, **STORE**, **ADD**, **SUB**, **MUL**, **MOV**, simulate target code generation for the expression:

$(a - b) * (c + d) - e$

Show the complete sequence of target instructions generated. Also explain the **runtime storage management** and **stack frame layout** used during the computation.

Given the three-address code sequence:

t1 = a + b,

t2 = t1 + 0,

t3 = t2 * 1,

if t3 == 0 goto L1,

t4 = t3 + t3

RUBRICS FOR CALCULATION

Simulation tools

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

SIMULATION TASK

Question 1: Peephole Optimization

Given three-address code:

```
t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y
```

Step-by-step Peephole Optimization

Step	Original Statement	Optimization Applied	Result
1	t1 = x * 1	Algebraic simplification ($a \times 1 = a$)	t1 = x
2	t2 = t1 + y	Copy propagation (t1 = x)	t2 = x + y
3	t3 = t2 - 0	Algebraic simplification ($a - 0 = a$) + copy propagation	t3 = x + y
4	t4 = t3 * 0	Algebraic simplification ($a \times 0 = 0$) → constant folding	t4 = 0
5	if t4 != 0 goto L1	Constant propagation (t4 is always 0) → condition is always false → dead code elimination	statement removed
6	t5 = y + y	Strength reduction ($a + a = 2 \times a$)	t5 = 2 * y

Optimized Instruction Sequence

```
t2 = x + y
t5 = 2 * y
```

Justification

- Algebraic simplification: multiplying by 1 ($t1 = x * 1$) and subtracting 0 ($t3 = t2 - 0$) do not change the value, so these operations are simplified to plain assignments/copies.
- Constant folding: $t4 = t3 * 0$ always evaluates to the constant 0 regardless of t3, so it is folded directly to $t4 = 0$ at optimization time.
- Dead code elimination: since t4 is always 0, the condition " $t4 \neq 0$ " can never be true, so the conditional jump to L1 is unreachable and is removed entirely, along with the now-unused t3 and t4 computations.
- Strength reduction: $t5 = y + y$ is rewritten as $t5 = 2 * y$ (or equivalently a left shift), which is cheaper on many target machines.
- The final optimized sequence contains only two useful instructions instead of six, with all redundant temporaries and unreachable control transfer eliminated.

Question 2: Basic Blocks and Control Flow Graph

Given intermediate code:

```
(1) x = a + b
```

```

(2) y = x * c
(3) if y > 50 goto 6
(4) z = y - d
(5) goto 7
(6) z = y + d
(7) w = z * 2

```

Rules Used to Identify Leader Statements

- Rule 1: The first statement of the program/block is a leader.
- Rule 2: Any statement that is the target of a conditional or unconditional goto is a leader.
- Rule 3: Any statement that immediately follows a conditional or unconditional goto statement is a leader.

Applying the Rules

Statement	Leader?	Rule Applied
(1) x = a + b	Yes	Rule 1 – first statement
(2) y = x * c	No	—
(3) if y > 50 goto 6	No	—
(4) z = y - d	Yes	Rule 3 – follows conditional goto (3)
(5) goto 7	No	—
(6) z = y + d	Yes	Rule 2 – target of goto in (3); also Rule 3 (follows goto in (5))
(7) w = z * 2	Yes	Rule 2 – target of goto in (5)

Basic Blocks

Block	Statements	Contents
B1	(1) – (3)	x = a + b ; y = x * c ; if y > 50 goto 6
B2	(4) – (5)	z = y - d ; goto 7
B3	(6)	z = y + d
B4	(7)	w = z * 2

Control Flow Graph (Edges)

- B1 → B2 (fall-through, taken when the condition "y > 50" is false)
- B1 → B3 (branch taken when the condition "y > 50" is true)
- B2 → B4 (unconditional jump, goto 7)
- B3 → B4 (fall-through into the next block)

Structure: B1 is the entry block with two successors (B2 and B3), both of which converge into the common exit block B4.

Question 3: DAG Construction and Code Generation

Given basic block:

```

p = a + b
q = a + b
r = p * q
s = a + b
t = r + s

```

DAG Construction

- Leaf node N1 represents variable a.
- Leaf node N2 represents variable b.
- Node N3 = N1 + N2 represents the expression a + b. Since p, q and s are all computed as a + b, all three labels (p, q, s) are attached to this single shared node N3 — the expression is evaluated only once.
- Node N4 = N3 * N3 represents p * q, labeled r.
- Node N5 = N4 + N3 represents r + s, labeled t.

Common Sub-expressions Identified

p, q and s are all equal to the expression a + b. Since a and b do not change between these assignments, q and s are common sub-expressions of p; the DAG merges all three into one node, eliminating two redundant additions.

Optimized Target Code (from the DAG)

```
T1 = a + b          // computes p, reused for q and s
T2 = T1 * T1        // r = p * q
T3 = T2 + T1        // t = r + s
```

Simple Code Generator – Register Allocation (Registers R0, R1)

Instruction	Target Code Emitted	Register Descriptor	Address Descriptor
T1 = a + b	MOV a, R0 ADD b, R0	R0 = {T1} R1 = { }	T1 → R0
T2 = T1 * T1 (r = p*q)	MOV R0, R1 MUL R0, R1	R0 = {T1} R1 = {T2}	T1 → R0 T2 → R1
T3 = T2 + T1 (t = r+s)	ADD R0, R1 STORE R1, t	R0 = { } (T1 dead) R1 = {T3, t}	t → R1 and memory

Explanation of Register Allocation Decisions

- T1 (= a + b) is needed three times (for p, q and s), so it is kept in register R0 for the entire computation instead of being recomputed or reloaded from memory — this directly mirrors the shared node in the DAG.
- R1 is used to hold each new intermediate result (T2, then T3); once a value in a register is no longer needed by any later instruction (e.g., T1 after computing T3), the register is considered free and can be reused.
- Because the DAG exposed the reuse of a + b, only three target instructions plus two supporting moves are needed instead of five separate additions, and only two registers are required throughout.

Question 4: Simple Code Generator with Two Registers (R0, R1)

Given code:

```
t1 = a + b
t2 = c * d
t3 = t1 - t2
t4 = e + f
t5 = t3 / t4
```

Instruction-wise Code Generation

Instruction	Target Code Emitted	Register Descriptor (after)	Address Descriptor (after)
t1 = a + b	MOV a, R0 ADD b, R0	R0 = {t1} R1 = { }	t1 → R0
t2 = c * d	MOV c, R1 MUL d, R1	R0 = {t1} R1 = {t2}	t1 → R0 t2 → R1
t3 = t1 - t2	SUB R1, R0	R0 = {t3} R1 = { } (t2 dead)	t3 → R0
t4 = e + f	MOV e, R1 ADD f, R1	R0 = {t3} R1 = {t4}	t3 → R0 t4 → R1
t5 = t3 / t4	DIV R1, R0 STORE R0, t5	R0 = {t5} (t4 dead) R1 = { }	t5 → R0 and memory

Discussion

- Each instruction is generated using the getReg algorithm: an operand already in a register is reused, and a free register is allocated for a new result whenever one is available.
- R0 holds t1 while R1 computes t2 in parallel; once t3 = t1 - t2 is computed, t2 becomes dead so R1 is freed and immediately reused for t4.
- The entire five-statement sequence is generated using only two registers, with no spills to memory required except the final STORE of the result t5.

Question 5: RISC Target Code Generation

Expression to evaluate: $(a - b) * (c + d) - e$

Available instructions: LOAD, STORE, ADD, SUB, MUL, MOV

Three-Address Code

```
t1 = a - b
t2 = c + d
t3 = t1 * t2
t4 = t3 - e
```

Target Instruction Sequence

```
LOAD  R1, a
LOAD  R2, b
SUB   R1, R1, R2      ; R1 = a - b = t1
LOAD  R2, c
LOAD  R3, d
ADD   R2, R2, R3      ; R2 = c + d = t2
MUL   R1, R1, R2      ; R1 = t1 * t2 = t3
LOAD  R2, e
SUB   R1, R1, R2      ; R1 = t3 - e = t4 (final result)
STORE result, R1      ; store final value to memory
```

The sequence uses nine target instructions and three registers (R1, R2, R3), loading each operand once and reusing R1 as the running accumulator for the final result.

Runtime Storage Management

At run time, memory used by a program is organized into three main areas:

- Static/Code area – holds the compiled instructions and any statically allocated (global) data at fixed addresses, determined at compile time.
- Stack – holds an activation record (stack frame) for each active procedure call; it grows when a procedure is called and shrinks when it returns, and stores parameters, local variables, and temporaries.
- Heap – holds dynamically allocated data whose size or lifetime is not known at compile time.

For the expression $(a - b) * (c + d) - e$ evaluated inside a single procedure with no further calls, the operands a, b, c, d, e are fetched from the current activation record (or global storage) into registers, and the temporaries t1 - t4 are held entirely in registers R1-R3, so no extra stack or heap space is required. If the number of live temporaries exceeded the available registers, the compiler would spill the excess temporaries into local-variable slots within the current stack frame.

Stack Frame (Activation Record) Layout

Region (high address → low address)	Purpose
Actual parameters	Values/addresses passed to this procedure by the caller
Return address	Address in the caller to resume execution after this procedure returns
Control link (dynamic link)	Pointer to the caller's activation record
Access link (static link)	Pointer used to access non-local variables in lexically enclosing scopes
Saved registers	Register values saved on entry and restored before returning
Local variables	Storage for the procedure's own local variables (a, b, c, d, e if local)
Temporaries	Spill slots for temporaries (t1-t4) that do not fit in available registers

The Frame Pointer (FP) gives a fixed reference for addressing parameters and locals by constant offset, while the Stack Pointer (SP) marks the current top of the frame; together they allow each activation record to be created on procedure entry and discarded on return.